MODÜL 4

İlişkisel Veritabanı

Table • Row • Column • Primary Key • Foreign Key • Relationships • Normalization • Schema • ER Diagram

Modül hedefi: Bir veri analisti olarak verinin sadece nasıl sorgulanacağını değil, neden o şekilde modellendiğini anlayabilmek. Bu modülün sonunda e-ticaret sistemi için doğru tabloları, anahtarları ve ilişkileri tasarlayabilir; kötü veri modelini fark edebilir ve analitik sorguların neden bazen hatalı sonuç verdiğini açıklayabilirsin.

Öğreneceğin alan	Pratik çıktı
Temel veritabanı kavramları	Tablo, satır, sütun ve veri tiplerini doğru yorumlama
PK / FK	Kayıtların benzersizliği ve tablolar arası bağlantı
Relationship türleri	1-1, 1-N, N-N ilişkilerini ayırt etme
Normalization	Tekrarı ve güncelleme problemlerini azaltan tasarım
Schema & ER diagram	E-ticaret veri modelini görselleştirme
Analitik bakış	JOIN hatalarını, double counting ve grain problemlerini fark etme

1. İlişkisel Veritabanı Nedir?

İlişkisel veritabanı, veriyi birbiriyle ilişkili tablolar halinde saklayan yapıdır. Her tablo belirli bir varlığı veya olayı temsil eder. Örneğin customers müşterileri, orders siparişleri, products ürünleri temsil edebilir.

> Bir veri analisti için kritik nokta şudur: tablo isimlerini bilmek yetmez; her tablonun grain seviyesini, yani bir satırın neyi temsil ettiğini bilmek gerekir.

Table, Row, Column

Kavram	Anlamı	E-ticaret örneği
Table	Aynı tür varlık veya olayların tutulduğu yapı	orders
Row	Tek bir kayıt	order_id = 101 olan sipariş
Column	Kaydın bir özelliği	order_date, customer_id, amount
Cell	Satır ve sütunun kesişimi	101 numaralı siparişin 1.250 TL tutarı

Bir tabloya bakarken ilk sorular:

- Bir satır tam olarak neyi temsil ediyor?
- Her satır benzersiz mi? Benzersizliği hangi sütun sağlıyor?
- Başka tablolar bu tabloya hangi alan üzerinden bağlanıyor?
- Bu tablonun olay tablosu mu, boyut tablosu mu olduğuna dair ipucu var mı?
- Aynı gerçek dünya bilgisinin başka bir tabloda tekrar saklanma ihtimali var mı?

2. Primary Key (PK) ve Foreign Key (FK)

Primary Key, bir tablodaki her satırı benzersiz olarak tanımlayan sütun veya sütun kombinasyonudur. Foreign Key ise başka bir tablodaki primary key değerine referans veren alandır.

customers	orders
customer_id (PK)	order_id (PK)
full_name	customer_id (FK)
email	order_date
city	total_amount

Burada orders.customer_id alanı, customers.customer_id alanına bağlanır. Böylece sipariş tablosunda müşterinin adını veya şehir bilgisini tekrar tekrar tutmak zorunda kalmazsın.

Primary key özellikleri

• Benzersiz olmalıdır.

• NULL olmamalıdır.

• Mümkün olduğunca değişmemelidir.

• Tek sütunlu veya composite key olabilir.

Composite Key nedir?

Bazen tek sütun benzersizliği sağlamaz. Örneğin order_items tablosunda bir siparişte aynı ürün yalnızca bir kez bulunacak şekilde tasarım varsa (order_id, product_id) birlikte primary key olabilir.

Bir PK aynı zamanda FK olabilir mi?

Evet. Özellikle 1-1 ilişkilerde mümkündür. Örneğin users tablosundaki user_id, user_profiles tablosunda hem PK hem FK olabilir. Böylece her kullanıcı için en fazla bir profil satırı tutulur.

3. Relationship Türleri

İlişkiler, bir tablodaki kayıtların diğer tablodaki kayıtlarla nasıl eşleştiğini ifade eder.

İlişki	Açıklama	Örnek
1 - 1	A tablosundaki bir kayıt B tablosunda en fazla bir kayda bağlanır	user - user_profile
1 - N	Bir kayıt diğer tabloda birçok kayıtla ilişkili olabilir	customer - orders
N - N	Her iki tarafta da çoklu eşleşme vardır	orders - products

N-N ilişki neden doğrudan kurulmaz?

Bir siparişte birden fazla ürün olabilir ve aynı ürün birçok siparişte bulunabilir. Bu nedenle orders ile products arasına order_items gibi bir junction/bridge tablo konur.

order_items
order_item_id (PK)
order_id (FK)
product_id (FK)
quantity
unit_price

Analitik risk: N-N ilişkileri doğru modellenmezse JOIN sonrasında satırlar çoğalabilir. Bu da revenue, customer count veya order count gibi KPI’ların olduğundan yüksek hesaplanmasına yol açabilir.

4. Grain: Bir Satır Neyi Temsil Ediyor?

Veri analistlerinin en kritik modelleme kavramlarından biri grain seviyesidir. Grain, bir tablodaki tek satırın anlamıdır.

Tablo	Grain
customers	1 satır = 1 müşteri
orders	1 satır = 1 sipariş
order_items	1 satır = bir siparişteki bir ürün satırı
daily_sales	1 satır = 1 gün + 1 mağaza özeti

Neden önemli?

• orders ile order_items JOIN edildiğinde bir sipariş birden fazla satıra çıkabilir.

• orders.total_amount alanını JOIN sonrası doğrudan SUM etmek duplicate revenue üretebilir.

• COUNT(*) ile COUNT(DISTINCT order_id) aynı şeyi ölçmez.

• Dashboard KPI’ları yanlış grain üzerine kurulursa yönetim yanlış karar verebilir.

Mini örnek

Bir siparişte 3 ürün varsa orders tablosunda sipariş 1 satırdır; order_items ile JOIN edildiğinde 3 satıra genişler. Sipariş toplamı her satıra taşınırsa 500 TL’lik sipariş yanlışlıkla 1.500 TL olarak toplanabilir.

5. Normalization Mantığı

Normalization, veriyi mantıklı tablolara bölerek tekrar eden bilgiyi azaltmayı, veri bütünlüğünü artırmayı ve insert/update/delete anomalilerini önlemeyi amaçlar.

Kötü tasarım örneği

order_id	customer_name	customer_city	product_name	category	qty
101	Ayşe Kaya	İstanbul	Kulaklık	Elektronik	1
102	Ayşe Kaya	İstanbul	Mouse	Elektronik	2
103	Mehmet Can	Ankara	Defter	Kırtasiye	3

Sorunlar:

• Ayşe Kaya’nın şehir bilgisi her siparişte tekrar eder.

• Şehir değişirse birden fazla satır güncellenmek zorunda kalır.

• Ürün kategorisi tekrar tekrar saklanır.

• Müşteri hiç sipariş vermediyse müşteri kaydı tutulamayabilir.

1NF - First Normal Form

Her hücre atomik olmalı; aynı hücrede virgülle ayrılmış çoklu değerler tutulmamalıdır. Örneğin product_ids = "5,8,12" yerine order_items tablosu kullanılmalıdır.

2NF

Özellikle composite key kullanılan tablolarda non-key alanlar anahtarın tamamına bağlı olmalıdır. Bir alan composite key’in sadece bir parçasına bağlıysa ayrı tabloya taşınmalıdır.

3NF

Non-key alanların başka non-key alanlara bağımlılığı azaltılır. Örneğin product_id üzerinden category_name belirleniyorsa category bilgisinin doğru yerde modellenmesi gerekir.

6. Insert, Update ve Delete Anomalileri

Anomali	Ne olur?	Örnek
Insert	Bir veriyi eklemek için ilgisiz başka veri gerekebilir	Sipariş olmadan yeni ürün eklenememesi
Update	Aynı bilgi birçok yerde güncellenir	Müşteri şehrini 20 sipariş satırında değiştirmek
Delete	Bir kaydı silerken başka bilgi de kaybolur	Son siparişi silince müşteri bilgisinin de yok olması

Normalization bu anomalilerin riskini azaltır. Ancak veri analitiğinde aşırı normalizasyon da sorguları gereksiz karmaşık hale getirebilir. Bu yüzden operasyonel sistemler ve analitik sistemler aynı modelleme yaklaşımını kullanmak zorunda değildir.

OLTP vs analitik düşünce

OLTP / operasyonel sistem	Analitik sistem
Transaction odaklı	Analiz odaklı
Daha normalize	Genellikle daha denormalize
Insert/update performansı kritik	Okuma ve aggregation kritik
Sipariş oluşturma, ödeme alma	Dashboard, KPI, trend analizi

7. Schema Nedir?

Schema, veritabanındaki tabloların, sütunların, veri tiplerinin, anahtarların ve ilişkilerin yapısal tanımıdır. Bir analist bir veritabanına ilk kez girdiğinde önce schema'yı anlamalıdır.

Örnek e-ticaret schema

Tablo	Temel sütunlar	Amaç
customers	customer_id, name, email, city, signup_date	Müşteri boyutu
orders	order_id, customer_id, order_date, status, total_amount	Sipariş başlığı
order_items	order_item_id, order_id, product_id, quantity, unit_price	Sipariş ürün detayları
products	product_id, product_name, category_id, cost	Ürün boyutu
categories	category_id, category_name	Kategori bilgisi
payments	payment_id, order_id, payment_method, paid_amount	Ödeme olayları

Analistin schema okuma sırası

- İş problemini belirle: satış mı, müşteri mi, ürün mü?
- Fact/olay niteliğindeki ana tabloyu bul.
- Primary key'i ve grain seviyesini belirle.
- İhtiyaç duyulan dimension tablolarını belirle.
- Foreign key bağlantılarını doğrula.
- JOIN sonrasında satır çoğalması olup olmadığını test et.

8. ER Diagram (Entity Relationship Diagram)

ER diagram, veritabanındaki varlıkları ve aralarındaki ilişkileri görsel olarak gösterir. Veri analisti için ER diagram, hangi tabloların nasıl JOIN edileceğini anlamanın en hızlı yollarından biridir.

Basitleştirilmiş e-ticaret modeli

CUSTOMERS	1	ORDERS
customer_id PK	→	order_id PK
name		customer_id FK
city		order_date
		total_amount
		↓ 1:N
PRODUCTS	N ←	ORDER_ITEMS
product_id PK		order_item_id PK
product_name		order_id FK
category_id FK		product_id FK
cost		quantity

ER diagram okurken sembollerin biçimi kullanılan araca göre değişebilir; fakat temel sorular değişmez: hangi tablo parent, hangisi child, kardinalite nedir ve foreign key nerede tutuluyor?

9. Referential Integrity ve Constraints

Referential integrity, foreign key değerlerinin referans verdiği parent tabloda gerçekten bulunmasını sağlamaya yönelik ilkedir.

Constraint	Amaç
PRIMARY KEY	Satır benzersizliği
FOREIGN KEY	Tablolar arası referans bütünlüğü
NOT NULL	Alan boş bırakılamaz
UNIQUE	Değerler tekrar edemez
CHECK	Belirli kurala uyan değerler kabul edilir
DEFAULT	Değer verilmezse varsayılan değer kullanılır

Örneğin orders.customer_id = 999 ise customers tablosunda 999 numaralı müşteri yoksa bu bir orphan record olabilir. Analiz sırasında bu tür kayıtlar JOIN ile kaybolabilir veya veri kalitesi sorununa işaret edebilir.

ON DELETE davranışları

• RESTRICT: bağlı kayıt varsa parent silinemez.

• CASCADE: parent silininces child kayıtlar da silinir.

• SET NULL: parent silininces foreign key NULL yapılır.

• Bu davranışların analitik veri üzerinde ciddi sonuçları olabilir; geçmiş kayıtların korunması çoğu iş senaryosunda önemlidir.

10. Gerçek Vaka: E-Ticaret Veritabanı Tasarla

Senaryo: Bir e-ticaret şirketi müşteriler, siparişler, ürünler ve ödemeler üzerinde dashboard oluşturmak istiyor. Aynı müşteri birçok sipariş verebilir; bir siparişte birçok ürün bulunabilir; ürünler kategorilere aittir; bir siparişin birden fazla ödeme denemesi olabilir.

Önerilen tablolar

Tablo	PK	Önemli FK	Grain
customers	customer_id	-	1 satır = 1 müşteri
orders	order_id	customer_id	1 satır = 1 sipariş
order_items	order_item_id	order_id, product_id	1 satır = 1 sipariş ürün satırı
products	product_id	category_id	1 satır = 1 ürün
categories	category_id	-	1 satır = 1 kategori
payments	payment_id	order_id	1 satır = 1 ödeme denemesi

İş sorusu: "Kategori bazında net satış ve benzersiz sipariş sayısı nedir?"

Analitik yol:

• orders → order_items: order_id üzerinden bağlanır.

• order_items → products: product_id üzerinden bağlanır.

• products → categories: category_id üzerinden bağlanır.

• Order count için COUNT(DISTINCT orders.order_id) düşünülür.

• Revenue hesabının hangi tabloda hangi grain'de bulunduğu kontrol edilir.

• İade/iptal siparişlerin status filtresiyle dışlanıp dışlanmayacağı iş tanımına göre belirlenir.

Önemli: Model doğru olsa bile KPI tanımı yanlışsa analiz yine yanlış olur. Veritabanı modelleme ile iş metriği tanımı birlikte düşünülmelidir.

11. Veri Analistinin Sık Yaptığı Modelleme Hataları

Hata	Sonuç	Kontrol
Grain’i bilmeden JOIN	Duplicate satırlar	JOIN öncesi/sonrası row count karşılaştır
PK’nin benzersiz olduğunu varsaymak	Yanlış count ve eşleşme	COUNT(*) vs COUNT(DISTINCT PK)
INNER JOIN’i otomatik kullanmak	Eşleşmeyen kayıtların kaybı	LEFT JOIN ile kayıp oranını kontrol et
N-N ilişkiyi fark etmemek	KPI şişmesi	Bridge tablo ve cardinality kontrolü
NULL FK değerlerini görmezden gelmek	Eksik segmentler	NULL oranını raporla
Revenue grain’ini karıştırmak	Double counting	Sipariş/ürün seviyesini ayır

Analitik kontrol listesi

- Her tablonun primary key’i nedir?
- Her tablonun grain’i nedir?
- Foreign key’lerde NULL veya yetim kayıt var mı?
- İlişki 1-N mi yoksa N-N mi?
- JOIN sonrası row count ne kadar değişti?
- Aggregation öncesi duplicate oluştu mu?
- İş metriği hangi tablonun hangi seviyesinde hesaplanmalı?

12. Mülakat Soruları

1. Primary key ile unique key arasındaki fark nedir?

Primary key tablo kimliğini temsil eder ve NULL kabul etmez; unique constraint ise benzersizlik sağlar ve DBMS davranışına göre NULL kuralları farklı olabilir.

2. Foreign key neden kullanılır?

Tablolar arasındaki referans bütünlüğünü sağlamak ve child kaydın geçerli bir parent kayda referans vermesini kontrol etmek için.

3. 1-N ilişki örneği ver.

Bir müşteri birçok sipariş verebilir; her sipariş bir müşteriye aittir.

4. N-N ilişki nasıl modellenir?

Araya bridge/junction tablo eklenir. Orders ve products için order_items tipik örnektir.

5. Normalization neden yapılır?

Veri tekrarını ve insert/update/delete anomalilerini azaltmak, veri bütünlüğünü artırmak için.

6. Grain ne demektir?

Bir tablodaki tek satırın temsil ettiği en düşük iş seviyesi.

7. JOIN sonrası revenue neden iki katına çıkabilir?

1-N veya N-N ilişki nedeniyle header-level revenue alanı birden fazla detay satırına çoğalabilir.

8. INNER JOIN ile LEFT JOIN farkı analitik açıdan neden önemlidir?

INNER JOIN eşleşmeyen kayıtları atar; LEFT JOIN ana tablonun tüm satırlarını korur. Kayıp veri ve coverage açısından sonuç farklılaşır.

9. PK aynı zamanda FK olabilir mi?

Evet. Özellikle 1-1 ilişkide child tablonun PK’si parent PK’ye FK olarak bağlanabilir.

10. ER diagram ne işe yarar?

Varlıkları, anahtarları ve kardinaliteleri görsel olarak gösterir; doğru JOIN yolunu anlamayı kolaylaştırır.

13. Uygulama Alıştırmaları

• Alıştırma 1: customers, orders ve order_items tabloları için her tablonun grain seviyesini yaz.

• Alıştırma 2: Bir öğrencinin birçok derse, bir dersin birçok öğrenciye ait olduğu sistemi modelle. Hangi bridge tablo gerekir?

• Alıştırma 3: Tek tabloda customer_name, city, order_id, product_name, category tutulan yapının en az üç problemini bul.

• Alıştırma 4: orders tablosunda order_id alanının duplicate çıktığını düşün. Analiz üzerindeki olası etkileri yaz.

• Alıştırma 5: products tablosundaki category_name bilgisini categories tablosuna ayır ve PK/FK ilişkisini tanımla.

• Alıştırma 6: 100.000 orders satırı ile JOIN sonrası 280.000 satır oluştu. Bunun normal olup olmadığını anlamak için hangi kontrolleri yaparsın?

• Alıştırma 7: Bir user_profile tablosunda user_id alanını hem PK hem FK olarak tasarla. Bunun hangi ilişkiyi zorladığını açıkla.

Beklenen çıktı

Kendi e-ticaret ER diagramını çiz: customers, orders, order_items, products, categories ve payments tablolarını kullan. Her tablo için PK, FK ve grain bilgisini ekle. Sonra "aylık kategori geliri" metriğinin hangi JOIN yoluyla hesaplanacağını yaz.

14. Portfolyo Mini Projesi: E-Ticaret DB Modeli

Proje amacı: Gerçek bir veri analisti gibi sıfırdan ilişkisel veri modeli tasarlamak ve modelin analitik kullanımını açıklamak.

Teslimat

• ER diagram görseli.

• Her tablo için: amaç, grain, PK, FK, önemli sütunlar.

• En az 5 relationship açıklaması.

• Normalization öncesi kötü tablo örneği ve düzeltilmiş model.

• En az 5 business question ve bu soruların hangi tablolarla çözüleceği.

• JOIN sonrası duplicate risklerini açıklayan kısa not.

• GitHub README: problem, model, varsayımlar, analitik kullanım.

Örnek business questions

• Aylık toplam revenue nedir?

• En yüksek revenue üreten kategoriler hangileri?

• Müşteri başına sipariş sayısı nedir?

• Ortalama sipariş başına ürün adedi nedir?

• Ödeme başarısızlık oranı nedir?

Bu projeyi tamamladığında SQL JOIN konularına geçiş çok daha kolay olur. Çünkü artık sorgunun sadece syntax'ını değil, tabloların neden o şekilde bağlandığını biliyor olursun.

Modül 4 tamamlandı → Sonraki modül: SQL Temel